Vibe Coding Competition
Study Guide

You and your teammate use AI to build a bot in Python. At the deadline your code freezes. Then your bot fights other bots, alone, live on the big screen. This guide explains the whole game and exactly what you will do on the day.

SUNDAY 18 OCTOBER | CIVIL AV AUDITORIUM · NED UNIVERSITY | TEAMS OF TWO | ANY AI TOOL | PYTHON 3.9+



A real match from the tournament show. Four bots, one arena. Nobody is controlling them: every move comes from the code each team wrote.

YOU BUILD

One Python file, `my_bot.py`. Each turn your bot looks at the game and picks one action.

YOU USE

Any AI tool you like: Claude, ChatGPT, Gemini, Cursor and others. Free or paid. Free plans work.

YOU HAVE

About 75 minutes to build, from 9:15 to 10:30 AM. Then your code is frozen.

YOU WIN BY

Finishing high in every map. Your bot plays several maps against the same three bots in its group.

01 EVENT DAY

What you will do, step by step

Read this part twice. Knowing the order of the day saves you more time than any trick.

Before the day

- [ ] Install **Python 3.9 or newer**. Open a terminal and type `python --version` to check.
- [ ] Charge your laptop and bring the charger.
- [ ] Install a code editor, for example VS Code or Cursor.
- [ ] Log in to the AI tool you plan to use, on the laptop you will bring.
- [ ] Learn how to open a terminal inside a folder on your laptop.
- [ ] Read this guide with your teammate. Decide who types and who tests.

On the day

8:45 AM UNTIL 9:00

Report at the Civil AV Auditorium

Find a seat with your teammate. **You get your team ID here.** Write it down. You need it to submit your bot.

9:00 AM

Scan the QR code and download the starter kit

- The organizers show a QR code on screen. It opens the starter kit repo.
- Download it (**Code** → **Download ZIP** on GitHub) and unzip it. If you know Git, `git clone` works too.
- Open a terminal inside the kit folder and run these two commands:

```
python check.py
python play.py
```

TERMINAL

Both should work straight away. If one fails, raise your hand. Fix this before you write any code.

9:15 AM

The contest starts: build your bot

- Give your AI the kit. The Vibe coding section shows how.
- Edit `my_bot.py`. It is the only file you submit.
- Repeat the build loop: change, check, play, look, change again.

~10:00 AM

Run the admission test

```
TERMINAL
python conformance.py
```

This is the same test the organizers run. **If it fails, your bot is rejected.** Run it early so you still have time to fix problems. Keep a copy of your last version that passed.

10:30 AM
DEADLINE

Email your bot. Code freeze.

Send one email to the submission address shared on the day, with:

- your `my_bot.py` file,
- your team name,
- your team ID.

After 10:30 nobody edits code. No AI, no human control. Your bot is on its own.

10:45 AM
TO 11:00

Bots are checked, then the games begin

The organizers run every bot through the admission test. When all bots are verified and in place, the tournament starts live on the big screen. Watch your bot play.

Same day

Results on screen are tentative

The organizers still review the submitted code after the tournament. A result shown on the day can change after that review.

Sat 24 Oct
9:00 AM ONWARDS

Closing ceremony, Main Auditorium

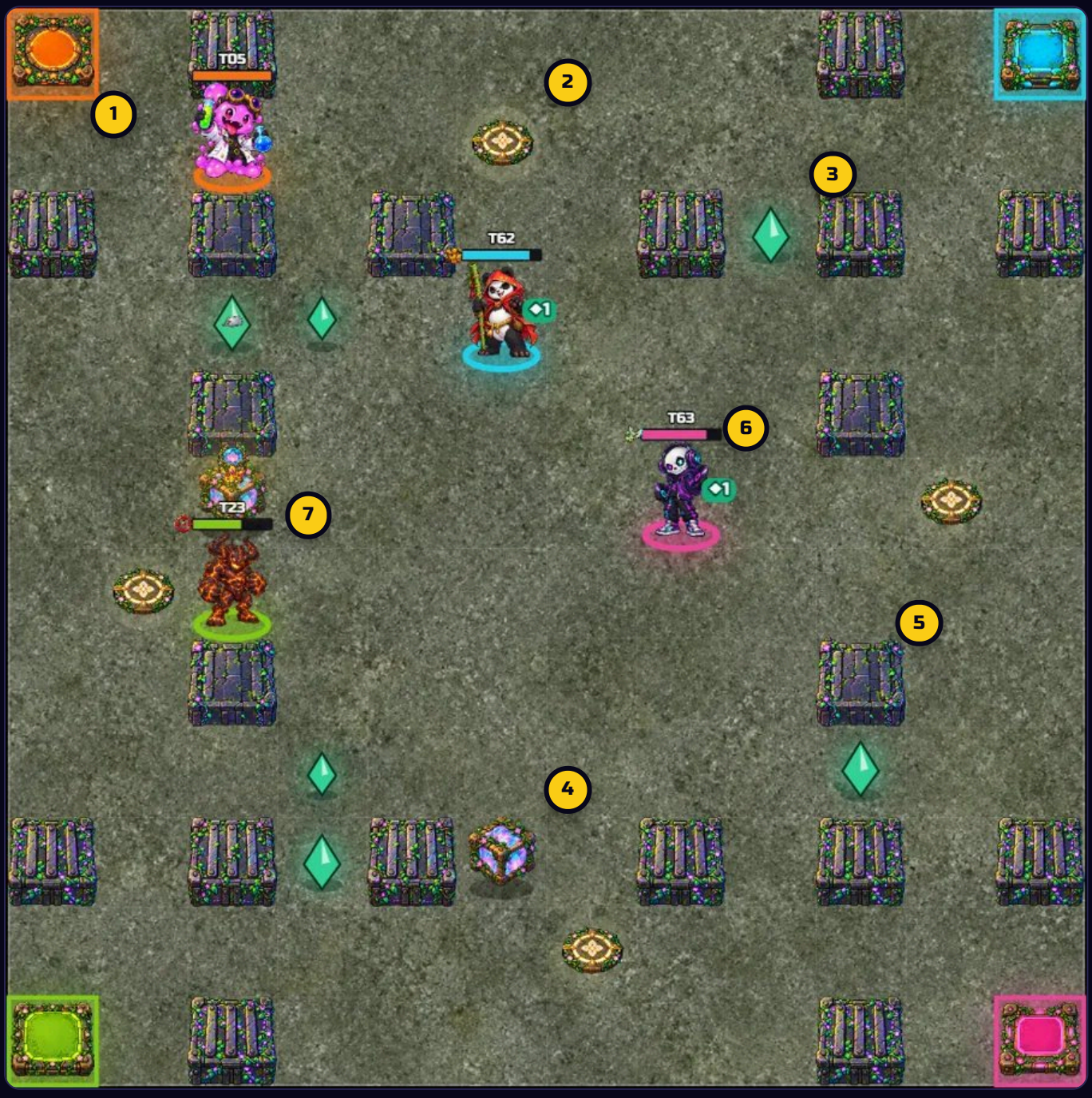
Final results are announced. Awards and prizes are given here.

02 THE GAME

Four bots, one arena

Every match puts four bots in a walled arena for 40 turns. Each bot tries to score the most points. Nobody is knocked out for good: a bot that dies comes back a few turns later.

4	12×12	40	×2	200 ms
BOTS PER MATCH	ARENA, WITH WALLS	TURNS PER MATCH	POINTS IN THE LAST 8 TURNS	TO DECIDE EACH TURN



1 **Your base**
Each bot starts in one corner. Your corner changes from map to map. Standing on your base banks your cargo.

2. **Uplink pad**
   There are four, one on each side. Carry the Core onto a pad to plant it.
3. **Resource**
   Collect it, carry it home, and bank it for points.
4. **Crate**
   Open it to get an item, like a railgun or a medpack.
5. **Wall**
   Walls block movement, shots and sight. The layout changes every match.
6. **A bot**
   The bar above it shows its health.
7. **A bot carrying the Core**
   It is marked, so every bot on the board can see it.

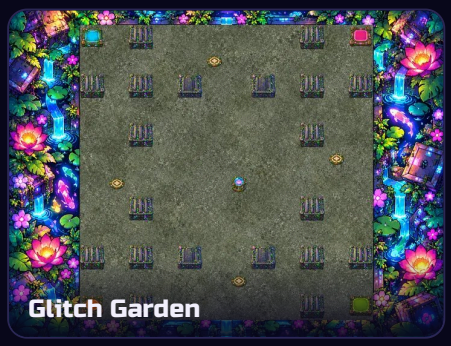
Things that are always true

- **Every corner is fair.** The walls are rotated four ways around the centre, so each corner faces exactly the same walls. Resources and crates also arrive in fair groups of four.
- **You only see nearby things.** Your bot sees 3 cells around it, and only where no wall blocks the line. This is called **fog of war**.
- **Some things are always visible:** the whole wall layout, the Core, the uplink pads, crates, smoke, airstrike markers, and any marked bot.
- **You see your own score and your rank** (1st to 4th). You never see another bot's score.
- **Dying never takes away points you already scored.** It costs you what you were carrying, your killing streak, and a few turns.

Eight map themes, one set of rules

Each map in the show uses one of eight themes. A theme changes only the look. The rules, the numbers and the walls work the same on every theme.









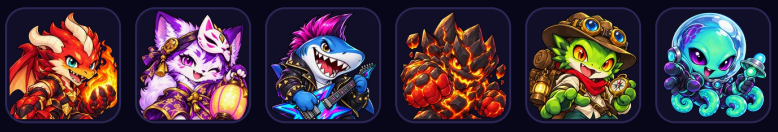








Every team also gets a character in the show. Like the themes, characters are only for the screen. They do not change how your bot plays.



03 YOUR TURN

Look, decide, act

Every turn, the game calls one function in your bot: `decide(view)`. The `view` is everything your bot can see right now. Your function returns **one action**.

> **ALL FOUR BOTS MOVE AT THE SAME TIME**
> Every bot decides from the same picture of the board. Then all the actions happen together. You never see what the others chose until the next turn.

Moving

- Your bot moves one cell per turn, in **8 directions**: `NORTH`, `SOUTH`, `EAST`, `WEST`, `NORTHEAST`, `NORTHWEST`, `SOUTHEAST`, `SOUTHWEST`.
- Diagonal steps count as one move. So the distance between two cells is the bigger of the two gaps: across or down.
- Walking into a wall or another bot is allowed, but you do not move. Your energy is still spent.
- **Walking off the edge of the board is illegal.**
- If two bots try to step onto the same cell, neither moves.
- Nothing can slip diagonally between two walls that touch at a corner. Not a bot, not a shot, not a line of sight.

Health and energy

- Your bot has **70 health** and **50 energy**.
- Every action costs energy. You get **+5 energy back every turn** (+10 in Overtime), up to 50.

Every action and its cost

ACTION	CODE	ENERGY	WHAT IT DOES
Move	`Action.move("EAST")`	1	Step one cell.
Attack	`Action.attack(bot_id)`	5	Hit a bot next to you, diagonals included, for 16–24 damage. It lands even if the target walks away this turn.
Shoot	`Action.shoot("NORTH")`	6	Fire in a straight line up to 4 cells. Hits the first bot on the line for 10–14. Walls stop it. It lands after everyone moves.
Collect	`Action.collect(id)`	2	Pick up the resource, crate or Core you are standing on.
Defend	`Action.defend()`	2	Take half damage this turn.
Scan	`Action.scan()`	8	See 3 cells further on your next turn.
Ability	`Action.ability(...)`	10	Use your special ability. See abilities.
Use item	`Action.use_item(...)`	4	Use the item you are holding. See items.

> **ILLEGAL ACTIONS COST YOU**
>
> An illegal action loses the turn. Examples: not enough energy, a target too far away, a move off the board, an ability still recharging. **One illegal action in the admission test gets your bot rejected.** So your bot must check every action against the view before it returns it.

Death and respawn

- At 0 health your bot is killed. It leaves the board and comes back **3 turns later** at its own base, with full health and energy.
- Everything it carried **spills on the floor** as one pile. Anyone can collect it. A held item drops as a crate.
- Its killing streak goes back to 0. **Banked points are never lost.**
- **Spawn protection:** for 2 turns after coming back, your bot takes no damage. Protection ends early if it attacks, shoots, collects, or uses an ability or item.
- Every base sits in a walled pocket with two exits, so nobody can shoot into a spawn from far away.

Marked bots

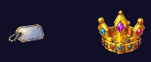
Some bots are **marked**. Every other bot can see a marked bot anywhere on the board, even through walls. A bot is marked while it:

MARK	WHY
`CROWN`	It has the highest score, so it wears the crown.
`CORE`	It is carrying the Core.
`PLANTER`	Its planted Core is counting down.
`CARGO`	It carries 60 or more cargo value.
`ON_FIRE`	It has a killing streak of 3 or more.
`LOUD`	It attacked, shot, or fired a big weapon last turn. Fighting is loud.

04 SCORING

Three ways to score

The bot with the most points when the 40 turns end wins the map. Points come from three places. You can use one, two or all three.



Kills

Bring an enemy to 0 health. A kill pays **+40** plus bonuses.

Part of each kill waits on the floor as a **tag**. Walk over it to collect the rest.



Cargo

1. Collect resources. They go into your hold.
2. Carry them home.
3. Stand on your base. Everything you carry is **banked** and scores its value.



The Core

1. Pick up the Core from the centre.
2. Carry it onto an uplink pad. It plants itself.
3. If it survives 3 turns, you score **+60**.

OVERTIME: THE LAST 8 TURNS PAY DOUBLE

From turn 32, **every point in the table below counts twice**: kills, bonuses, banked cargo, uplinks and defuses. Energy also comes back twice as fast.

Every way to earn points

EVENT	POINTS
Kill: +25 at once, +15 when you collect its tag	+40
Kill a bot you already killed in the last 8 turns (all of it is in the tag)	+15
First Blood: the first kill of the match	+20
Crown kill: the victim wore the crown. +15 more for each extra map in a row it had won	+30
Shutdown: the victim had a streak of 2 / 3 / 4 / 5 or more	+10 / 20 / 30 / 40
On Fire: you kill while your own streak is 3 or more	+10
Multi-kill: each extra kill you make in the same turn	+20
Revenge: you kill the bot that last killed you	+15
Nemesis: you kill the bot that killed you most last map	+15
Clutch: you make the kill while on 15 health or less	+15
Assist: you hurt the victim in the last 3 turns, but someone else got the kill	+10
Blast: your own bomb, airstrike or orbital made the kill	+10
Shove kill: a bot you shoved died to a blast within 2 turns. You also share the kill	+10
Tag confirmed: you collect the tag of your own kill	+15
Tag denied: you collect someone else's tag, so the killer never gets the rest	+10
Cargo banked at your base	its value
Core uplinked: your planted Core survives its countdown	+60
Core defused: shared by every enemy next to the Core when the defuse completes	+40

One kill can earn several rows at once. Your role adds a few rows of its own. Every point your bot earns appears in the match log, with the reason.

How kills work

- **Kill Confirmed.** When a bot dies, it drops a tag where it fell. A bot standing on a tag at the end of a turn picks it up. If a killer picks it up, that is **confirmed** (+15). If anyone else does, even the victim after respawning, that is **denied** (+10). Tags disappear after 6 turns.
- **Shared kills.** Every bot that dealt damage on the killing turn gets the kill.
- **Streaks.** Your streak is the number of kills since you last died. At a **3-kill streak** you get a free radar sweep: you see every enemy for 3 turns. At a **5-kill streak** you get an airstrike in your item slot.
- **The crown.** At the start of each turn, the bot with the highest score wears the crown. It is marked for everyone. Nobody learns by how much it leads.
- A bot killed by its own bomb gives nobody any points.

How cargo works

- Collected resources are **not** points yet. They only score once you stand on your own base.
- Your hold carries **4 resource units**. When it is full, you must bank before you can collect again.
- **Hot pickup:** collecting while another bot is within 2 cells loads the resource at 1.3 times its value. A spilled pile never gets this bonus again.
- Cargo still in your hold when the match ends scores nothing.
- The match starts with 4 resources. Most resources arrive later in **supply waves** during the match.

How the Core works

1. **Spawn.** The Core appears on turn 8 on one of the four centre cells.
2. **Pick up.** Stand on it and collect `"CORE"`. You are now marked. If you die, the Core drops where you fell.
3. **Plant.** Step onto any uplink pad while carrying it. It plants itself at the end of that turn.
4. **Uplink.** If it survives 3 turns, the planter scores +60, even if the planter died in the meantime.
5. **Defuse.** If an enemy stays within 1 cell of the planted Core at the end of 2 turns in a row, it is defused. Every enemy next to it scores +40, and the uplink never happens.
6. **Respawn.** After an uplink or a defuse, the Core comes back at the centre 4 turns later.

The Core's position and status are always visible to every bot.

If two bots finish a map on the same score

The tie is broken in this order: more kills, then fewer deaths, then more cargo banked, then more resources collected.

05 ABILITIES & ITEMS

Everything on the board

This part explains what each piece does and what to watch out for. It does not tell you which one to pick. That choice is yours.

Special abilities: pick one for the whole match

You set your ability once, in your code, with `ABILITY = "DASH"` (or `None`). Using it costs 10 energy. Then it recharges for 6 turns.



DASH

`Action.ability(direction="EAST")`

Move 2 cells in a straight line, in any of the 8 directions.

WATCH Both cells must be free of walls and bots.



BLINK

`Action.ability(direction="NORTH")`

Teleport 3 cells in a straight line, over walls and bots.

WATCH The landing cell must be free.



SHIELD

`Action.ability()`

Cut incoming damage by 70% for this turn.

WATCH It only lasts one turn.



BOMB

`Action.ability()`

Drop a bomb on your own cell. 3 turns later it explodes: 45 damage in the centre, 25 one cell away.

WATCH It hurts you too. Everyone nearby can see it. One live bomb at a time.



SHOVE

`Action.shove(bot_id)`

Push an enemy next to you one cell straight away from you.

WATCH If a wall, a bot or the edge is behind it, it is slammed for 15 damage instead.

BOMB TIMING

A bomb explodes at the **start** of a turn, before any bot decides. So the shield or defend that protects you is the one you used on the turn before, when the bomb shows `turns_until_detonation: 1`. The same is true for airstrikes and orbitals. Blasts also set off other bombs inside their area.

Crates and items

Crates drop in groups of four during the match. Everyone can see them. Stand on one and collect it to get the item inside. Using an item costs 4 energy.

- You hold **one item at a time**. Opening a crate while holding an item replaces the old one.
- The item is chosen when you open the crate. **The lower your rank, the likelier a power item** (railgun or airstrike). Likelier, never certain.
- Each match uses only 4 of the 5 item types. The list is in `view["item_pool"]`.

- Rivals can see that you hold an item, but not which one.



RAILGUN

`Action.use_item(direction="WEST")`

Fires to the first wall or the edge and hits **every** bot on the line for 25.

WATCH It is loud, and it lands after everyone moves.



AIRSTRIKE

`Action.use_item(x=5, y=6)`

Aim at any cell, even in fog. It lands two turns later: 40 in the centre, 25 one cell away.

WATCH Everyone sees the marker at once. It hurts you if you stand in it.



MEDPACK

`Action.use_item()`

Heal 40 health, never above your maximum.



SMOKE

`Action.use_item()`

A 3×3 cloud on your cell for 3 turns. Bots inside are hidden from anyone more than 1 cell away.

WATCH Marks still show through smoke.



RADAR

`Action.use_item()`

See every enemy on the board for 3 turns.

The legendary drop

Once per match, a gold crate lands near the centre. Everyone gets a warning 2 turns before, in `view["legendary_incoming"]`. It holds one of three legendary items.



LEGENDARY

ORBITAL

`Action.use_item(x=6, y=6)`

A bigger airstrike: radius 2, 60 in the centre, 30 at the edge. Lands 2 turns later.

WATCH Everyone can see where it will land.



LEGENDARY

OVERSHIELD

`Action.use_item()`

For 3 turns, the next hit that would kill you leaves you on 1 health instead.



LEGENDARY

REAPER

`Action.use_item()`

For 5 turns, every kill you make pays its kill points twice.

Events

Every match gets each of these two events once, at some point during the match.

EVENT	WHAT HAPPENS	HOW LONG
BLOOD_MOON	Attack, shoot, railgun and shove-slam damage ×1.5. Blasts are not boosted.	6 turns
RESOURCE_RUSH	Four extra resources drop in the inner arena.	Instant. The resources stay.

Taunts

Any action can carry one emote for the match feed, like `Action.defend().taunt("GG")`. The allowed list: `GG`, `NICE`, `OOPS`, `TOO_SLOW`, `WATCH_THIS`, `RIP`, `BOOM`, `HELLO`, `THANKS`, `CATCH_ME`. A taunt never changes what the action does. It is just for the crowd.

06 ROLES & SHOP

Roles, credits and the shop

Before every map, your bot secretly picks a role and buys upgrades. Nobody else can see your role or what you bought.

The four roles

A role makes one way of scoring pay more, and costs you somewhere else. You can also pick no role (`None`): no bonus and no cost.



Hunter

+ +10 per kill. Rampage: +5 per kill for each kill already on your streak.
– Banks cargo at 0.85×. Whoever kills you gets +25.



Hauler

+ +1 cargo space. Banking 3 units at once pays 1.25×, 4 or more pays 1.5×.
– Kills pay 10 less.



Runner

+ +10 per uplink, plus 10 more for each earlier uplink this map. Ability recharges 2 turns faster.
– 5 less maximum health.



Warden

+ +20 per defuse. Guard: +20 for killing a Core carrier, a live planter, or anyone within 2 cells of a planted Core.
– Can never pick up the Core. Banks at 0.9×. 1 less cargo space.

Role bonuses double in Overtime, like everything else.

Credits

- Every bot starts a series with **4 credits**.
- After every map you get **+3**, plus a placement bonus: **+0** for 1st, **+1** for 2nd, **+2** for 3rd, **+3** for 4th.
- You also get **+1 for every map in a row you have not won**, up to +2.
- Unspent credits carry over. You can hold at most **10**.

The shop

ITEM	PRICE	EFFECT
ARMOR	2	+15 maximum health. Rivals only ever see your health.
POCKETS	2	+1 cargo space.
SCOPE	2	+1 vision range.
QUICK_RESPAWN	3	Come back 1 turn sooner.
OVERCLOCK	3	Ability recharges 2 turns faster (never below 3 turns).
KIT_RAILGUN	3	Start the map holding a railgun.
KIT_MEDPACK	2	Start holding a medpack.
KIT_SMOKE	2	Start holding a smoke.
KIT_RADAR	2	Start holding a radar.

Each item can be bought once per map. You can buy at most one `KIT_`, because you have one item slot.

How your bot buys

Write a `buy(self, shop)` function. The game calls it once before every map. `shop` holds your credits, the prices, the roles, the map number and your own past placements. It never holds anything about a rival.

```
def buy(self, shop):
    # shop["credits"], shop["prices"], shop["map"], shop["your_placements"]
    return {"role": "HAULER", "items": ["POCKETS"]}
```

> **A BAD BUY IS NEVER ILLEGAL**
>
> An unknown name, a duplicate, two kits, more than you can afford, a crash or a timeout simply buys nothing. You still play the map. Return `None` to keep your class's `ROLE` and buy nothing.

What carries from one map to the next

- The last map's winner starts the next map **wearing the crown**.
- A bot that won several maps in a row carries a bigger crown bounty (+15 per extra win).
- Your **nemesis** is whoever killed you most last map. Killing it pays +15.
- Credits carry over.
- Nothing else. Every map starts with fresh health, energy, scores and an empty board.

07 THE TOURNAMENT

How the tournament works

Bots play in groups of four. A group plays a series of maps. The best bots in each group move on to the next round.

Rank points

In a series, what counts is **where you finish each map**, not your score.

3	2	1	0
FOR 1ST PLACE	FOR 2ND PLACE	FOR 3RD PLACE	FOR 4TH PLACE

So every map matters, even one you cannot win. Finishing 2nd in every map beats winning once and finishing last four times.

The rounds

ROUND 1	ROUND 2	SEMIFINAL	GRAND FINAL
5 maps	**5 maps**	**7 maps**	**10 maps**
The top bot of each group advances.	The top 2 of each group advance.	The top 2 of each group advance.	One group of four. The winner is champion.

This is the plan for a full field. If the number of teams changes, the organizers confirm the final format on the day.

Fair play built in

- Your starting corner rotates across the series.
- The groups are drawn in advance.

If teams are level on rank points

The tie is broken in this order. The first rule that separates the teams decides.

1. **Head-to-head:** how many times each team finished above the other tied teams in this series.
2. **Map wins:** more maps finished 1st.
3. **Total score** across the series.
4. **Core captures.**
5. **Banked cargo value.**

If a rule separates some of the tied teams but not all, the ones still level start again from head-to-head among just themselves. The standings screen in the show names the rule and the numbers for every tie.

When a bot breaks

- A bot that crashes, takes too long, or returns a bad action **loses that turn**, not the match.
- After 3 failures in a row, it is switched off for the rest of that match.
- A broken bot only hurts its own team.

Results

The show displays results live on the day. They are **tentative** until the organizers finish reviewing the submitted code. Final results, awards and prizes are announced at the **closing ceremony on Saturday 24 October, from 9:00 AM, in the Main Auditorium**.



Map 2 of 5 on Glitch Garden. The small table at the top shows the series so far, with the advance line.

08 VIBE CODING

Build your bot with AI

You do not need to be an expert coder. You need to explain your idea clearly, test often, and read what the tools tell you.

What is in the starter kit

FILE	WHAT IT IS FOR
`my_bot.py`	Your bot. **The only file you edit and submit.** It starts as a simple legal bot.
`docs/RULES.md`	Every rule and number in the game.
`docs/BOT_SDK_CONTRACT.md`	What your bot receives each turn and what it must return.
`check.py`	Quick test: is my bot valid?
`play.py`	Play matches and see what happened.
`conformance.py`	The full admission test the organizers run.
`sparring.py`	Chaos bots that act at random. They find crashes in your bot.
`engine/`, `vibe_sdk/`	The real game engine. You never edit these, but your AI can read them.

Two ways to work. Both are fine.

A code editor with AI

Open the **whole kit folder** in the editor. The AI can read every file, edit `my_bot.py` and often run the commands for you.

Cursor · Claude Code · Codex · Antigravity · VS Code + Copilot

A browser chat

Attach `docs/RULES.md`, `docs/BOT_SDK_CONTRACT.md` and `my_bot.py`. Copy the new code back into `my_bot.py` yourself, then run the commands. Free plans work.

ChatGPT · Claude · Gemini · Grok

The build loop

1 **Change**
Ask your AI for one small change.

2 **Check**
Run `python check.py`. Is it still valid?

3 **Play**
Run `python play.py --seeds 2`
How does it do across 20 maps?

4 **Look**
Run `python play.py` and read the turn log. Is it doing what you meant?

5

Repeat

Tell the AI what you saw, and pick the next change.

TERMINAL

```
python play.py                          # one match, full turn log
python play.py --seed 42                # a different map
python play.py --seeds 20               # 20 maps and a summary
python play.py --vs sparring:ChaosShove # play against a chaos bot
python check.py                         # still valid?
python conformance.py                   # the admission test
```

TEST AGAINST DIFFERENT OPPONENTS

By default, `play.py` puts your bot against three copies of itself. The tournament uses maps and opponents you have never seen. Use `--seeds` to see many maps, and `--vs` to play against other bots, including ones you write yourself.

The one rule about your code

- `decide(view)` must return **one action** within **200 milliseconds**. So must `buy(shop)`. Aim for tens of milliseconds: the tournament machine is busier than your laptop.
- Your bot must do **exactly the same thing** every time it gets the same input. No `time.time()`, no unseeded `random`, no reading files, no internet. A seeded generator like `random.Random(7)` is fine.
- Your bot keeps one object for the whole match, so `self.anything = ...` remembers things between turns.
- Your class must be called `MyBot`. Put every helper inside `my_bot.py`. A second file works on your laptop but does not exist at the tournament.
- Import only `vibe_sdk` and the Python standard library. Some modules are blocked, like `os`, `sys`, `subprocess`, `socket`, `pickle` and `threading`, and some calls, like `eval`, `exec` and `open`. `check.py` tells you if you break this.

Example strategy sketches

These are deliberately very simple. They are not good plans and they are not recommendations. They only show how an idea becomes a list of rules, and how rules become code. Your own idea is the whole point of the competition.

The Wanderer

1. Pick a random direction that stays on the board.
2. Move that way.

The Walker

1. If you see an enemy, walk toward it.
2. If it is next to you, attack it.
3. Otherwise, walk to the middle.

The Picker

1. If you stand on a resource, collect it.
2. If your hold is full, walk home.
3. Otherwise, walk to a resource you can see.

The Fetcher

1. If the Core is on the ground, walk to it and collect it.
2. If you carry it, walk to an uplink pad.
3. Otherwise, walk to the middle.

From sketch to code: The Walker

Here is The Walker as a complete `my_bot.py`. It passes `check.py` and `conformance.py`.

```python
from vibe_sdk import DIRECTION_DELTAS, Action, Bot


class MyBot(Bot):
    ABILITY = None
    ROLE = None

    def decide(self, view):
        me = view["self"]
        enemies = view["visible_enemies"]

        if enemies:
            target = enemies[0]
            distance = max(abs(target["x"] - me["x"]), abs(target["y"] - me["y"]))

            # Next to an enemy and enough energy: attack it.
            if distance == 1 and me["energy"] >= 5:
                return Action.attack(target["id"])

            # Otherwise walk toward it.
            return Action.move(self.step_toward(me, target["x"], target["y"]))

        # Nobody in sight: walk to the middle.
        board = view["board"]
        return Action.move(self.step_toward(me, board["width"] // 2, board["height"] // 2))

    def step_toward(self, me, target_x, target_y):
        dx = (target_x > me["x"]) - (target_x < me["x"])
        dy = (target_y > me["y"]) - (target_y < me["y"])
        if (dx, dy) == (0, 0):
            dx = 1 if me["x"] == 0 else -1
        for name, delta in DIRECTION_DELTAS.items():
            if delta == (dx, dy):
                return name
        return "EAST"
```

Now look at what `python play.py` printed when we ran it:

TERMINAL

```
T00  YOU → MOVE to (10,1)
T01  YOU → MOVE to (9,2) blocked
T02  YOU → MOVE to (9,2) blocked
T03  YOU → MOVE to (9,2) blocked
```

The bot walks into a wall every turn. That is legal, so `check.py` passes. But the bot is stuck. **Valid does not mean good.** This is why you read the turn log, and then tell your AI what you saw.

Mistakes that break a bot

- Assuming something is in the view. Fog of war means most lists are often **empty**. Empty lists and missing things are normal.
- Not checking energy, range, board edges or ability cooldown before returning an action.
- Assuming an item exists. Each match uses only some items. Check `view["item_pool"]`.
- Assuming the walls are the same as last time. They change every match. Read `view["walls"]`.
- Using slow code. Searching the whole board many times per turn can pass 200 ms.
- Waiting until 10:25 to run `conformance.py`.

> **LET THE ENGINE DO THE GEOMETRY**
>
> `vibe_sdk.geometry` is the engine's own wall code: `line_clear`, `shot_path` and `step_open`. If your bot uses it, it always agrees with the engine about sight lines and shot paths.

09 PROMPTS

Prompts you can copy

Use these as a starting point. Replace the gold text with your own words. One small request at a time works better than one giant request.

1 Understand the game

```
Read docs/RULES.md and docs/BOT_SDK_CONTRACT.md. Do not write any code yet.
Explain the game to me in simple bullet points:
- what my bot sees each turn,
- every action it can return, with its energy cost,
- every way to score points.
Then list anything that would make an action illegal.
```

Use first. Read the answer together and check it against this guide.

2 Turn your idea into rules

```
Here is our idea for the bot: [describe your idea in a few sentences].
Turn it into a short, numbered list of if-then rules for decide(view), checked from top to bottom.
Do not write code yet. Ask me questions if anything is unclear.
```

Agree on the rules before any code exists. It is much easier to fix a rule than code.

3 Write the bot

```
Write these rules into my_bot.py: [paste your rules].
Requirements:
- Edit only my_bot.py. Keep the class name MyBot.
- Import only vibe_sdk and the Python standard library.
- Before returning any action, check energy, range, board edges and cooldown against the view.
- Treat every list in the view as possibly empty.
- Every possible path through decide() must return a legal Action.
- No unseeded randomness and no time-based code.
Give me the complete file.
```

Then run `python check.py`.

4 Fix an error

```
I ran [python check.py] and it printed this:
[paste the full output]
Explain the cause in one or two sentences. Then fix my_bot.py and give me the complete file.
```

Always paste the full output, not your summary of it.

5 Fix what you saw

```
I ran python play.py. Here is part of the turn log:
[paste the lines that show the problem]
I expected my bot to [what you wanted], but it [what it actually did].
Find out why and suggest one small change. Keep every legality check.
```

Describe what you expected and what really happened. That gives the AI a real problem to solve.

6 Compare two versions

```
Here are two summaries from python play.py --seeds 20.
Before the change: [paste]
After the change: [paste]
What changed? Did the change do what we wanted? What should we look at next?
```

One map can be luck. Twenty maps tell you more.

7 Pass the admission test

```
python conformance.py failed with this output:
[paste the full output]
Fix my_bot.py so it passes. Do not change what the bot is trying to do unless the fix
needs it.
Make sure decide() and buy() are fast, deterministic and can never return an illegal
action.
```

Use this as soon as the test fails. Then run it again.

8 Explain the code to me

```
Explain my_bot.py line by line in simple English.
For each rule in decide(), tell me which situation in the game triggers it.
```

You should understand your own bot. It also makes your next request clearer.

HABITS THAT HELP

- **Save a copy every time it passes.** For example, `my_bot_backup.py`. If a change breaks everything, go back.
- **One change at a time.** If you change five things and it gets worse, you will not know which one did it.
- **Do not trust code blindly.** Run it, read the log, and check it against the rules.
- **Split the work.** One teammate talks to the AI. The other runs the tests and reads the logs.
- **Remember to submit** `my_bot.py`, not your backup file.

10 CHEAT SHEET

Every number in one place

MATCH

Bots per match	**4**
Arena	**12 × 12**
Turns	**40**
Overtime starts	**turn 32**
Decision time	**200 ms**

YOUR BOT

Health	**70**
Energy (max)	**50**
Energy back per turn	**+5 (+10 OT)**
Vision	**3 cells**
Cargo space	**4 units**
Respawn	**3 turns**
Spawn protection	**2 turns**

ACTION COSTS

Move	**1**
Collect · Defend	**2**
Use item	**4**
Attack	**5**
Shoot	**6**
Scan	**8**
Ability	**10**

DAMAGE

Attack (range 1)	**16–24**
Shoot (range 4)	**10–14**
Railgun (whole line)	**25**
Bomb · centre / edge	**45 / 25**
Airstrike · centre / edge	**40 / 25**
Orbital · centre / edge	**60 / 30**
Shove slam	**15**
Defend · Shield	**−50% · −70%**
Blood Moon	**×1.5**

MAIN POINTS

Kill	**+40**
Repeat kill (8 turns)	**+15**
First Blood	**+20**
Crown kill	**+30**
Core uplink	**+60**
Core defuse	**+40**
Cargo banked	**its value**
Hot pickup	**×1.3**

TIMING

Core first appears	**turn 8**
Core countdown	**3 turns**
Defuse needs	**2 turns**
Ability recharge	**6 turns**
Tags last	**6 turns**

SERIES

Rank points	**3 / 2 / 1 / 0**
Starting credits	**4**
Credits per map	**+3**
Placement bonus	**0 / 1 / 2 / 3**
Credit cap	**10**

EVENT DAY

Reporting · team ID	**8:45 AM**
Kit QR on screen	**9:00 AM**
Contest starts	**9:15 AM**
Deadline · code freeze	**10:30 AM**
Games begin	**10:45–11:00**
Closing ceremony	**Sat 24 Oct**

The full rules are in `docs/RULES.md` in the starter kit. If this guide and the engine ever disagree, the engine decides.

11 FAQ

Questions

Can we use any AI tool?

Yes. Any AI tool, free or paid, while you build. Free plans work. After the 10:30 code freeze, your bot runs alone: it cannot call an AI or use the internet during a match.

Do we need to install anything except Python?

No. You need Python 3.9 or newer. The kit needs no `pip install`.

How big is a team?

Two people.

When do we get the starter kit?

On the day, at 9:00 AM. It is shared as a QR code on screen.

When do we get our team ID?

At reporting, from 8:45 to 9:00 AM. You need it in your submission email.

What exactly do we submit?

One email before 10:30 AM with your `my_bot.py` file, your team name and your team ID. The class inside must be called `MyBot`.

What if our bot fails the admission test?

It is rejected. That is why you run `python conformance.py` yourself, early. A bot that crashes, takes longer than 200 ms, behaves differently between runs, or makes even one illegal action fails.

Can we see the other teams' code or scores?

No. During a match your bot sees only its own score and its rank. Nobody sees another team's code.

Is the result on the day final?

No. It is tentative. Final results are announced at the closing ceremony on Saturday 24 October, from 9:00 AM, in the Main Auditorium.

Is there a best strategy?

No single strategy is right. Kills, cargo and the Core all score. How well a plan does depends on the other three bots, and you will not know who they are until the match.

Haven't registered yet?

Limited spots left. Grab a teammate and sign up for the Vibe Coding Competition at TechFest '26.

Sunday 18 October · Civil AV Auditorium · NED University

SCAN TO REGISTER